DSA MASTER SHEET

MAANG & Top-Tech Interview Ready

12 Topics · 50+ Patterns · 200+ Curated Problems

How to use	*Pick 1 pattern/day. Solve problems in order. Revisit starred problems weekly.*
Difficulty	*[Easy] [Medium] [Hard] shown beside each problem.*
Goal	*After finishing, you can crack any MAANG / FAANG interview.*

Table of Contents

Arrays & Strings

Foundation of every interview. Master traversal, manipulation, and in-place tricks.

Pattern: Two Pointers

Tip: Use when array is sorted or you need O(1) space. Eliminate nested loops.

- **[Easy]** Two Sum II (sorted input)
- **[Medium]** 3Sum
- **[Medium]** Container With Most Water
- **[Hard]** Trapping Rain Water
- **[Easy]** Remove Duplicates from Sorted Array
- **[Easy]** Valid Palindrome
- **[Medium]** Sort Colors (Dutch National Flag)

Pattern: Sliding Window

Tip: For subarray/substring problems. Expand right, shrink left on violation.

- **[Medium]** Longest Substring Without Repeating Chars
- **[Hard]** Minimum Window Substring
- **[Medium]** Longest Repeating Character Replacement
- **[Medium]** Max Consecutive Ones III
- **[Medium]** Fruit Into Baskets
- **[Medium]** Permutation in String
- **[Hard]** Sliding Window Maximum

Pattern: Prefix Sum / Difference Array

Tip: Precompute cumulative sums for range queries in O(1).

- **[Medium]** Subarray Sum Equals K
- **[Medium]** Product of Array Except Self
- **[Easy]** Range Sum Query (Immutable)
- **[Medium]** Continuous Subarray Sum
- **[Hard]** Count of Range Sum
- **[Medium]** Minimum Size Subarray Sum

Pattern: Kadane's / Max Subarray

Tip: Reset running sum when it goes negative. Track global max.

- **[Medium]** Maximum Subarray
- **[Medium]** Maximum Product Subarray
- **[Easy]** Best Time to Buy and Sell Stock
- **[Medium]** Circular Subarray Maximum Sum
- **[Medium]** Maximum Sum of Two Non-Overlapping Subarrays

Pattern: Matrix Traversal

Tip: Use direction arrays [dr, dc] for cleaner BFS/DFS on grids.

- **[Medium]** Spiral Matrix
- **[Medium]** Rotate Image
- **[Medium]** Set Matrix Zeroes
- **[Medium]** Search a 2D Matrix II
- **[Medium]** Game of Life
- **[Medium]** Word Search

Hashing & Maps

O(1) lookup power. Reduce O(n^2) brute force to O(n) with a map.

Pattern: Frequency Counting

Tip: Count occurrences, then query. Great for anagram/permutation problems.

- **[Medium]** Group Anagrams
- **[Medium]** Top K Frequent Elements
- **[Medium]** Find All Anagrams in a String
- **[Easy]** First Unique Character in a String
- **[Easy]** Word Pattern
- **[Easy]** Isomorphic Strings

Pattern: Complement / Two-Sum Pattern

Tip: Store seen values; check if complement exists before inserting.

- **[Easy]** Two Sum
- **[Medium]** 4Sum II
- **[Medium]** Pairs of Songs With Total Durations Divisible by 60
- **[Medium]** Subarray Sum Equals K
- **[Medium]** Count Number of Nice Subarrays

Pattern: Index as Hash / Cyclic Sort

Tip: When values in range [1,n], use index tricks to find missing/duplicate.

- **[Medium]** Find the Duplicate Number
- **[Easy]** Missing Number
- **[Medium]** Find All Duplicates in an Array
- **[Hard]** First Missing Positive
- **[Easy]** Set Mismatch

Linked Lists

Pointer manipulation. Always draw boxes and arrows before coding.

Pattern: Fast & Slow Pointers (Floyd's)

Tip: Detect cycles, find midpoints, or the k-th from end.

- **[Easy]** Linked List Cycle
- **[Medium]** Linked List Cycle II (entry point)
- **[Easy]** Middle of the Linked List
- **[Easy]** Happy Number
- **[Medium]** Reorder List
- **[Easy]** Palindrome Linked List

Pattern: Reversal

Tip: Master in-place reversal. Use prev, curr, next pointers.

- **[Easy]** Reverse Linked List
- **[Medium]** Reverse Linked List II (range)
- **[Hard]** Reverse Nodes in k-Group
- **[Medium]** Swap Nodes in Pairs
- **[Medium]** Rotate List

Pattern: Merge / Sort

Tip: Use dummy node trick to simplify edge cases at head.

- **[Easy]** Merge Two Sorted Lists
- **[Hard]** Merge K Sorted Lists
- **[Medium]** Sort List
- **[Medium]** Add Two Numbers
- **[Medium]** Partition List
- **[Medium]** Remove Nth Node From End

Stacks & Queues

Monotonic stack is a top interview pattern. Think: 'what's the next greater element?'

Pattern: Monotonic Stack

Tip: Maintain increasing or decreasing order. O(n) for next greater/smaller.

- **[Medium]** Daily Temperatures
- **[Medium]** Next Greater Element I & II
- **[Hard]** Largest Rectangle in Histogram
- **[Hard]** Maximal Rectangle
- **[Medium]** Sum of Subarray Minimums
- **[Medium]** Remove K Digits
- **[Medium]** 132 Pattern

Pattern: Valid Parentheses / Parsing

Tip: Push open brackets; on close, check top matches.

- **[Easy]** Valid Parentheses
- **[Medium]** Minimum Remove to Make Valid Parentheses
- **[Medium]** Decode String
- **[Medium]** Basic Calculator II
- **[Medium]** Evaluate Reverse Polish Notation
- **[Hard]** Longest Valid Parentheses

Pattern: Sliding Window + Monotonic Deque

Tip: Deque stores indices; front is max/min of current window.

- **[Hard]** Sliding Window Maximum
- **[Hard]** Shortest Subarray with Sum >= K
- **[Hard]** Max Value of Equation

Binary Search

Any monotone condition on a sorted/searchable space — binary search it.

Pattern: Classic Binary Search

Tip: Use lo + (hi-lo)//2. Decide which half to discard based on condition.

- **[Easy]** Binary Search
- **[Easy]** Search Insert Position
- **[Medium]** Find Minimum in Rotated Sorted Array
- **[Medium]** Search in Rotated Sorted Array
- **[Medium]** Find Peak Element
- **[Easy]** First Bad Version

Pattern: Binary Search on Answer

Tip: When asked for min/max feasible answer, binary search over the answer space.

- **[Medium]** Koko Eating Bananas
- **[Medium]** Minimum Number of Days to Make m Bouquets
- **[Hard]** Split Array Largest Sum
- **[Medium]** Capacity to Ship Packages Within D Days
- **[Hard]** Median of Two Sorted Arrays
- **[Medium]** Nth Root of a Number

Pattern: 2D / Matrix Binary Search

Tip: Treat matrix as 1D or use row + column binary search.

- **[Medium]** Search a 2D Matrix
- **[Medium]** Search a 2D Matrix II
- **[Medium]** Kth Smallest Element in a Sorted Matrix
- **[Hard]** Find Kth Smallest Pair Distance

Recursion & Backtracking

Explore all possibilities with pruning. Think: 'choose, explore, un-choose'.

Pattern: Subsets / Power Set

Tip: At each element: include or exclude. 2^n total subsets.

- **[Medium]** Subsets
- **[Medium]** Subsets II (with duplicates)
- **[Medium]** Letter Case Permutation
- **[Medium]** Count of Subsets with Given Sum

Pattern: Permutations

Tip: Swap elements to generate all orderings. Use visited[] or swapping.

- **[Medium]** Permutations
- **[Medium]** Permutations II (with duplicates)
- **[Medium]** Next Permutation
- **[Hard]** Permutation Sequence

Pattern: Combinations / N-Queens Style

Tip: Build candidates incrementally; prune invalid states early.

- **[Medium]** Combination Sum
- **[Medium]** Combination Sum II
- **[Medium]** Combination Sum III
- **[Hard]** N-Queens
- **[Hard]** Sudoku Solver
- **[Medium]** Word Search
- **[Medium]** Palindrome Partitioning
- **[Medium]** Generate Parentheses

Trees (Binary Tree & BST)

DFS and BFS are your two weapons. Understand preorder, inorder, postorder deeply.

Pattern: DFS Traversals

Tip: Preorder = process before children; postorder = process after. Both recursive and iterative.

- **[Easy]** Binary Tree Inorder Traversal
- **[Medium]** Binary Tree Level Order Traversal
- **[Medium]** Binary Tree Zigzag Level Order
- **[Medium]** Path Sum II
- **[Hard]** Binary Tree Maximum Path Sum
- **[Medium]** Flatten Binary Tree to Linked List
- **[Hard]** Serialize and Deserialize Binary Tree

Pattern: Tree Properties / Construction

Tip: Use inorder+preorder/postorder to reconstruct. Diameter = max(left_depth + right_depth).

- **[Easy]** Maximum Depth of Binary Tree
- **[Easy]** Diameter of Binary Tree
- **[Easy]** Balanced Binary Tree
- **[Easy]** Symmetric Tree
- **[Medium]** Construct BT from Preorder+Inorder
- **[Medium]** Lowest Common Ancestor of Binary Tree
- **[Medium]** Count Good Nodes in Binary Tree

Pattern: BST Properties

Tip: Inorder of BST is sorted. Validate using min/max bounds (not just parent).

- **[Medium]** Validate Binary Search Tree
- **[Medium]** Kth Smallest Element in a BST
- **[Medium]** Lowest Common Ancestor of BST
- **[Medium]** Delete Node in a BST
- **[Medium]** Insert into a BST
- **[Easy]** Convert Sorted Array to BST
- **[Medium]** BST Iterator

Heaps & Priority Queues

For top-K, K-th largest/smallest, or merging K sorted structures.

Pattern: Top K Elements

Tip: Min-heap of size K for top K largest. Max-heap for K smallest.

- **[Medium]** Kth Largest Element in an Array
- **[Medium]** Top K Frequent Elements
- **[Medium]** K Closest Points to Origin
- **[Medium]** Find K Pairs with Smallest Sums
- **[Easy]** Kth Largest Element in a Stream

Pattern: Merge K Sorted / Two Heaps

Tip: Use max-heap + min-heap to maintain median. Push to both, balance sizes.

- **[Hard]** Merge K Sorted Lists
- **[Hard]** Find Median from Data Stream
- **[Hard]** Sliding Window Median
- **[Hard]** IPO (capital scheduling)
- **[Medium]** Ugly Number II
- **[Medium]** Task Scheduler

Graphs

BFS = shortest path (unweighted). DFS = cycle detection, topology, components.

Pattern: BFS / Level-Order

Tip: Use a deque. Track visited before enqueue, not dequeue, to avoid reprocessing.

- **[Medium]** Number of Islands
- **[Medium]** Rotting Oranges
- **[Hard]** Word Ladder
- **[Medium]** Open the Lock
- **[Medium]** Minimum Knight Moves
- **[Medium]** Walls and Gates
- **[Medium]** 01 Matrix

Pattern: DFS / Connected Components

Tip: Mark visited in-place when possible. Count or label each component.

- **[Medium]** Number of Islands
- **[Medium]** Max Area of Island
- **[Medium]** Pacific Atlantic Water Flow
- **[Medium]** Course Schedule (cycle detection)
- **[Medium]** Clone Graph
- **[Medium]** Surrounded Regions

Pattern: Topological Sort

Tip: Kahn's (BFS indegree) or DFS post-order. For dependency/ordering problems.

- **[Medium]** Course Schedule II
- **[Hard]** Alien Dictionary
- **[Medium]** Sequence Reconstruction
- **[Medium]** Minimum Height Trees
- **[Hard]** Sort Items by Groups Respecting Dependencies

Pattern: Union-Find (Disjoint Set)

Tip: Path compression + union by rank = near O(1) per op. Great for dynamic connectivity.

- **[Medium]** Number of Connected Components
- **[Medium]** Redundant Connection
- **[Medium]** Accounts Merge
- **[Medium]** Graph Valid Tree
- **[Hard]** Making a Large Island

Pattern: Shortest Path (Dijkstra / Bellman-Ford)

Tip: Dijkstra for non-negative weights (heap + dist[]). Bellman-Ford for negative edges.

- **[Medium]** Network Delay Time
- **[Medium]** Cheapest Flights Within K Stops
- **[Medium]** Path With Maximum Probability
- **[Medium]** Find the City with Smallest Neighbors
- **[Hard]** Swim in Rising Water

Dynamic Programming

DP = recursion + memoization = optimal substructure + overlapping subproblems.

Pattern: 1D DP (Linear)

Tip: dp[i] depends on previous 1-2 states. Often replaces O(2^n) recursion.

- **[Easy]** Climbing Stairs
- **[Medium]** House Robber
- **[Medium]** House Robber II (circular)
- **[Medium]** Jump Game
- **[Medium]** Jump Game II (min jumps)
- **[Medium]** Decode Ways
- **[Medium]** Word Break

Pattern: 2D DP / Grid

Tip: dp[i][j] from dp[i-1][j] and dp[i][j-1]. Watch for boundary conditions.

- **[Medium]** Unique Paths
- **[Medium]** Minimum Path Sum
- **[Hard]** Edit Distance
- **[Medium]** Longest Common Subsequence
- **[Medium]** Interleaving String
- **[Medium]** Maximal Square
- **[Hard]** Dungeon Game

Pattern: Knapsack (0/1 and Unbounded)

Tip: 0/1: loop items outer, capacity inner, backward. Unbounded: forward inner loop.

- **[Medium]** Partition Equal Subset Sum
- **[Medium]** Target Sum
- **[Medium]** Coin Change
- **[Medium]** Coin Change II (ways)
- **[Medium]** Last Stone Weight II
- **[Medium]** Ones and Zeroes
- **[Hard]** Profitable Schemes

Pattern: Interval DP

Tip: dp[i][j] = best over all split points k in [i,j]. O(n^3).

- **[Hard]** Burst Balloons
- **[Hard]** Matrix Chain Multiplication
- **[Hard]** Minimum Cost to Cut a Stick
- **[Hard]** Strange Printer
- **[Hard]** Palindrome Partitioning II

Pattern: LIS / LCS Variants

Tip: LIS in O(n log n) with patience sorting. LCS is classic 2D DP.

- **[Medium]** Longest Increasing Subsequence
- **[Hard]** Russian Doll Envelopes
- **[Medium]** Longest Common Subsequence
- **[Medium]** Longest Palindromic Subsequence
- **[Medium]** Number of Longest Increasing Subsequences
- **[Hard]** Delete Columns to Make Sorted III

Pattern: DP on Trees / State Machine

Tip: DP on trees uses postorder DFS. State machines track which 'state' we're in.

- **[Medium]** House Robber III (on tree)
- **[Hard]** Binary Tree Cameras
- **[Medium]** Best Time to Buy/Sell Stock with Cooldown
- **[Medium]** Best Time to Buy/Sell Stock with TX fee
- **[Hard]** Best Time to Buy/Sell Stock III (2 tx)
- **[Hard]** Best Time to Buy/Sell Stock IV (k tx)

Tries (Prefix Trees)

Efficient prefix search. Build Trie from dictionary, then query in O(L) per word.

Pattern: Build & Search

Tip: Each node has children[26]. Insert word char by char; mark end of word.

- **[Medium]** Implement Trie (Prefix Tree)
- **[Medium]** Search Suggestions System
- **[Medium]** Longest Word in Dictionary
- **[Medium]** Replace Words
- **[Medium]** Design Add and Search Words

Pattern: Trie + DFS / Backtracking

*Tip: Combine Trie with grid DFS for multi-word search in O(M*4^L).*

- **[Hard]** Word Search II
- **[Hard]** Palindrome Pairs
- **[Hard]** Concatenated Words
- **[Medium]** Maximum XOR of Two Numbers (Bitwise Trie)

Greedy Algorithms

Prove your greedy choice is always safe. Sort first, then make local optimal picks.

Pattern: Sorting + Greedy

Tip: Sort by start, end, or ratio. Process in order to minimize cost/maximize coverage.

- **[Medium]** Non-overlapping Intervals
- **[Medium]** Merge Intervals
- **[Medium]** Meeting Rooms II
- **[Medium]** Insert Interval
- **[Medium]** Minimum Number of Arrows
- **[Medium]** Task Scheduler

Pattern: Greedy on Arrays

Tip: Make local optimal at each step. Common: jump game, gas station, candy.

- **[Medium]** Jump Game
- **[Medium]** Jump Game II
- **[Medium]** Gas Station
- **[Hard]** Candy
- **[Medium]** Partition Labels
- **[Medium]** Reorganize String

Bit Manipulation

Know your bit ops: AND, OR, XOR, shifts. XOR is your go-to for uniqueness problems.

Pattern: XOR Tricks

Tip: a^a=0, a^0=a. XOR all elements to cancel duplicates.

- **[Easy]** Single Number
- **[Medium]** Single Number II
- **[Easy]** Missing Number
- **[Medium]** Find the Duplicate Number
- **[Medium]** XOR Queries of a Subarray

Pattern: Bit Counting / Masks

Tip: n & (n-1) clears lowest set bit. Use masks to represent subsets.

- **[Easy]** Number of 1 Bits
- **[Easy]** Counting Bits
- **[Easy]** Reverse Bits
- **[Easy]** Power of Two
- **[Medium]** Subsets via Bitmask
- **[Medium]** Maximum XOR of Two Numbers

Problem-Solving Framework

1. Understand	Restate the problem. Identify input type, constraints, edge cases.
2. Pattern Match	Which topic does it belong to? Two pointers? DP? Graph?
3. Brute Force First	Write the O(n^2) or exponential solution. Then optimize.
4. Optimize	Apply the pattern: reduce time with hash map, sliding window, DP, etc.
5. Code Cleanly	Use meaningful variable names. Modularize with helpers.
6. Test	Dry run with given example → edge case (empty, single, max) → stress test.
7. Complexity	Always state time and space complexity before finishing.

Consistency beats intensity. Solve 2 problems a day, every day. 90 days = interview-ready.